

# Ejemplo 4 (XMLHttpRequest POST + Promesa): Crear un Post

## Objetivo del ejercicio

- Aprender a **enviar datos al servidor** usando `XMLHttpRequest` dentro de una **promesa**.
- Practicar el uso de **then, catch y finally** para manejar la respuesta y errores.
- Mostrar la respuesta en la pantalla usando **Bootstrap**.

---

## Código completo (HTML + Bootstrap + JS)

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejemplo 4 - XMLHttpRequest POST con Promesas</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">
</head>
<body class="container py-4">

  <h1 class="mb-4">Ejemplo 4: Crear un Post con XMLHttpRequest y
Promesas</h1>
  <p>Rellena los campos y envía un post a la API de prueba
<code>JSONPlaceholder</code>.</p>

  <form id="formPost" class="mb-3">
    <div class="mb-3">
      <label for="userId" class="form-label">User ID</label>
      <input type="number" class="form-control" id="userId"
value="1" required>
    </div>
    <div class="mb-3">
      <label for="titulo" class="form-label">Título del
Post</label>
      <input type="text" class="form-control" id="titulo"
placeholder="Título" required>
    </div>
    <div class="mb-3">
      <label for="contenido" class="form-
label">Contenido</label>
      <textarea class="form-control" id="contenido" rows="3"
placeholder="Escribe el contenido..." required></textarea>
    </div>
    <button type="submit" class="btn btn-primary">Enviar
Post</button>
  </form>

  <div id="resultadoPost" class="mt-3"></div>

</script>
```

```

    const formPost = document.getElementById('formPost');
    const resultadoPost =
document.getElementById('resultadoPost');

    // Función que devuelve una promesa para enviar POST
    function crearPost(data) {
        return new Promise((resolve, reject) => {
            const xhr = new XMLHttpRequest();
            xhr.open('POST',
'https://jsonplaceholder.typicode.com/posts', true);
            xhr.setRequestHeader('Content-Type',
'application/json; charset=UTF-8');

            xhr.onreadystatechange = () => {
                if (xhr.readyState === 4) {
                    if (xhr.status === 201) { // 201 = creado
                        const respuesta =
JSON.parse(xhr.responseText);
                        resolve(respuesta);
                    } else {
                        reject(`Error ${xhr.status}:
${xhr.statusText}`);
                    }
                }
            };

            // Convertimos el objeto a JSON antes de enviar
            xhr.send(JSON.stringify(data));
        });
    }

    // Evento submit del formulario
    formPost.addEventListener('submit', (e) => {
        e.preventDefault();

        const userId = document.getElementById('userId').value;
        const title = document.getElementById('titulo').value;
        const body = document.getElementById('contenido').value;

        resultadoPost.innerHTML = `<div class="alert alert-
info">Enviando post...</div>`;

        // Llamamos a la promesa
        crearPost({ userId, title, body })
            .then(respuesta => {
                resultadoPost.innerHTML = `
                <div class="alert alert-success">
                    Post creado con éxito!<br>
                    ID: ${respuesta.id}<br>
                    Título: ${respuesta.title}<br>
                    Contenido: ${respuesta.body}<br>
                    User ID: ${respuesta.userId}
                </div>`;
            })
            .catch(error => {
                resultadoPost.innerHTML = `<div class="alert
alert-danger">${error}</div>`;
            })
            .finally(() => {
                console.log("Petición POST finalizada");
            });
    });

```

```
    });  
</script>  
  
</body>  
</html>
```

---

## Explicación paso a paso

1. **Función `crearPost(data)`:**
    - o Envuelve un `XMLHttpRequest` POST dentro de una promesa.
    - o `resolve(respuesta)` se llama cuando la API devuelve **201 Created**.
    - o `reject(error)` se llama si hay fallo (cualquier otro status).
  2. **Evento `submit` del formulario:**
    - o Evita la recarga de página con `e.preventDefault()`.
    - o Obtiene los valores del formulario.
    - o Llama a `crearPost()` y maneja la respuesta con `.then()`, errores con `.catch()` y finalización con `.finally()`.
  3. **Enviar datos:**
    - o `xhr.setRequestHeader('Content-Type', 'application/json; charset=UTF-8')` indica que se enviará JSON.
    - o `xhr.send(JSON.stringify(data))` convierte los datos a formato JSON.
  4. **Visualización de resultados:**
    - o La respuesta de la API se muestra dentro de un `alert` de Bootstrap, incluyendo el `id` generado por la API.
- 

## Preguntas de comprensión para alumnos

1. ¿Por qué usamos `xhr.setRequestHeader('Content-Type', 'application/json')`?
  2. ¿Cuál es la diferencia entre un GET y un POST en AJAX?
  3. ¿Qué indica el código HTTP 201?
  4. ¿Por qué usamos `JSON.stringify()` al enviar los datos?
  5. ¿Cómo podrías validar los campos del formulario antes de enviar la petición?
- 

## Mejoras sugeridas para el alumno

1. Limpiar el formulario automáticamente después de enviar el post.
2. Añadir un **spinner de carga** mientras se envía la petición.
3. Mostrar los posts enviados en una **lista debajo del formulario**.
4. Permitir enviar posts a un **servidor local con Node.js + Express** en lugar de la API de prueba.
5. Manejar errores más detallados, por ejemplo, si el título o contenido está vacío.